LAPORAN TUGAS ILP

Implementasi Dependency RAW, WAR, dan WAW

Mata Kuliah	Instruction-Level Parallelism (ILP)
Nama	Wildan Jati Raga Putra Kusyana
NIM	51258057
Anggota 2	.....
Anggota 3	.....
Repository GitHub	<link href="https://github.com/wildankusyana-sudo/tugas-ilp-war-raw-waw" color="blue">https://github.co

Catatan: lengkapi nama dua anggota lain dan unggah kode program ke repository sebelum dikumpulkan.

1. Tujuan dan Konsep Dasar

Tugas ini membahas tiga jenis ketergantungan instruksi pada ILP. Dependency graph menunjukkan urutan instruksi yang harus dipertahankan agar hasil program benar.

RAW (Read After Write) adalah ketergantungan ketika instruksi berikutnya membaca register yang ditulis instruksi sebelumnya (true dependency). **WAR (Write After Read)** terjadi ketika instruksi berikutnya menulis register yang sebelumnya dibaca instruksi terdahulu (anti-dependency). **WAW (Write After Write)** terjadi ketika dua instruksi menulis register yang sama (output dependency).

2. Implementasi dan Dependency Graph

A. RAW

```
I1: R1 = R2 + R3
I2: R4 = R1 * R5

[I1: menulis R1] -- RAW (R1) --> [I2: membaca R1]
```

I1 menghasilkan R1 dan I2 menggunakan R1, sehingga I2 bergantung pada hasil I1.

B. WAR

```
I1: R1 = R2 + R3
I2: R2 = R4 + R5

[I1: membaca R2] -- WAR (R2) --> [I2: menulis R2]
```

I1 harus membaca nilai lama R2 sebelum I2 menimpa R2.

C. WAW

```
I1: R1 = R2 + R3
I2: R1 = R4 * R5

[I1: menulis R1] -- WAW (R1) --> [I2: menulis R1]
```

Kedua instruksi menulis R1. Urutan penulisan harus dipertahankan agar nilai akhir sesuai urutan program.

3. Program Simulasi Python

Simpan kode berikut sebagai **simulator_ilp_war_raw_waw.py** dan jalankan dengan Python 3.

```
def dependency_graph(instructions):
    edges = []
    for i in range(len(instructions)):
        for j in range(i + 1, len(instructions)):
            a, b = instructions[i], instructions[j]
            if a["write"] and a["write"] in b["read"]:
                edges.append((i+1, j+1, "RAW", a["write"]))
            if b["write"] and b["write"] in a["read"]:
                edges.append((i+1, j+1, "WAR", b["write"]))
            if a["write"] and a["write"] == b["write"]:
                edges.append((i+1, j+1, "WAW", a["write"]))
    return edges

cases = {
 "RAW": [
  {"text":"R1 = R2 + R3","read":{"R2","R3"},"write":"R1"},
  {"text":"R4 = R1 * R5","read":{"R1","R5"},"write":"R4"}],
 "WAR": [
  {"text":"R1 = R2 + R3","read":{"R2","R3"},"write":"R1"},
  {"text":"R2 = R4 + R5","read":{"R4","R5"},"write":"R2"}],
 "WAW": [
  {"text":"R1 = R2 + R3","read":{"R2","R3"},"write":"R1"},
  {"text":"R1 = R4 * R5","read":{"R4","R5"},"write":"R1"}]
}
for name, instructions in cases.items():
    print("\n", name)
    for n, inst in enumerate(instructions, 1):
        print(f"I{n}: {inst['text']}")
    for src, dst, kind, reg in dependency_graph(instructions):
        print(f"I{src} -> I{dst} [{kind}, register {reg}]")
```

4. Hasil yang Diharapkan

```
RAW: I1 -> I2 [RAW, register R1]
WAR: I1 -> I2 [WAR, register R2]
WAW: I1 -> I2 [WAW, register R1]
```

5. GitHub dan Kolaborasi

Repository kelompok: https://github.com/wildankusyana-sudo/tugas-ilp-war-raw-waw

Unggah file program dan laporan ke repository tersebut. Setiap anggota perlu melakukan commit dengan akun masing-masing dan kontribusi nyata. Contoh pesan commit: “Add RAW dependency example”, “Add WAR and WAW examples”, dan “Document simulator and report”.

6. Kesimpulan

RAW menjaga hubungan produsen-konsumen data; WAR menjaga pembacaan nilai lama sebelum register ditulis ulang; WAW menjaga urutan penulisan ke register yang sama. Dependency graph memvisualisasikan hubungan tersebut dan membantu mengidentifikasi batas pelaksanaan instruksi secara paralel.

Referensi: materi kuliah ILP dan notebook simulator yang diberikan dosen. Contoh ini bersifat edukatif dan perlu disesuaikan jika notebook dosen menggunakan sintaks khusus.